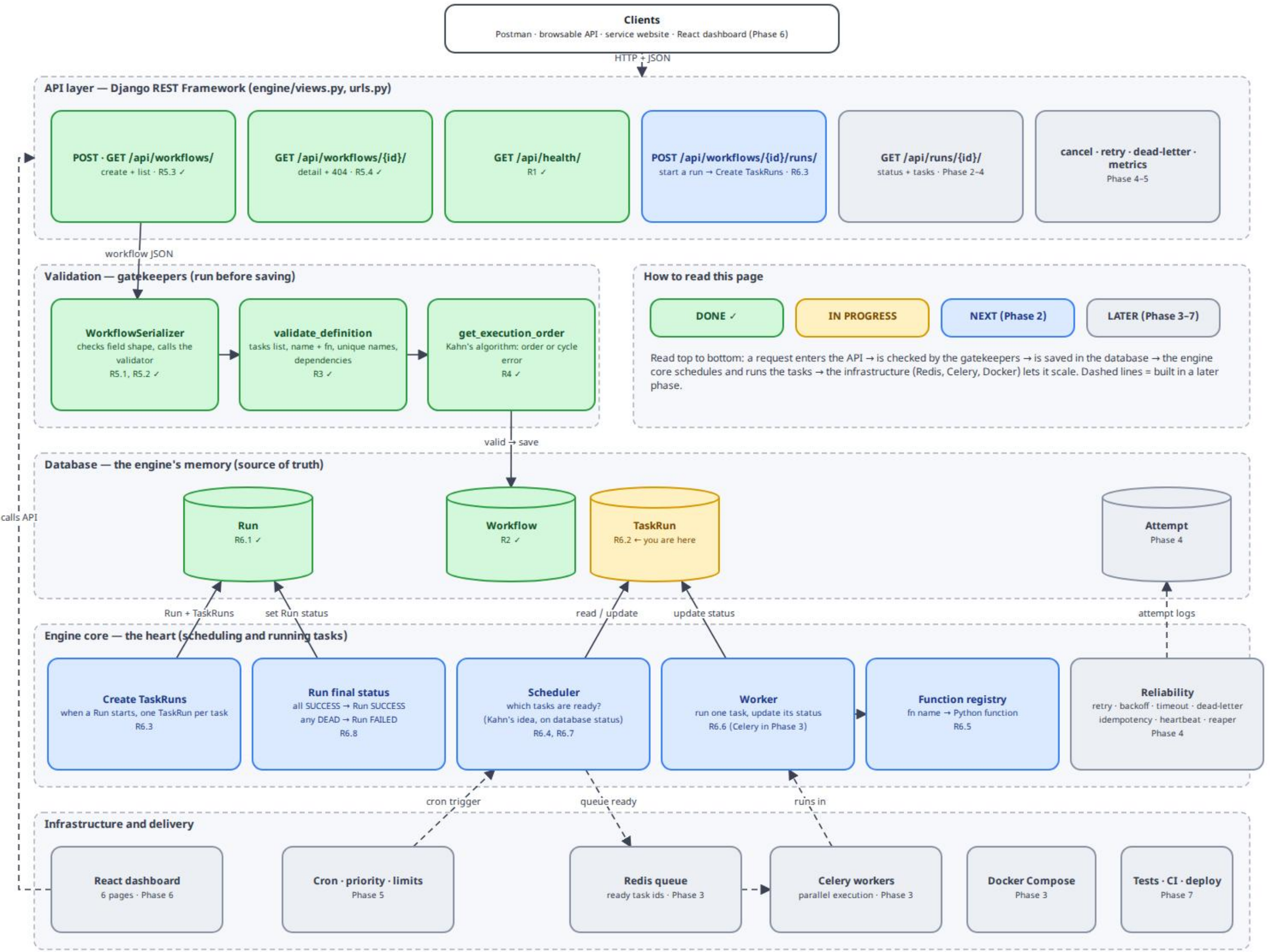
Workflow Orchestration Engine — Architecture: built vs left

R-numbers are your task tickets (R6.3 = ticket R6.3). Colour = status. Updated: Oct 2026



Task roadmap — every task, connected phase by phase

Each box = one ticket. Arrows show the order. Green done · amber now · blue next · grey later.

DONE ✓ | IN PROGRESS | NEXT | LATER

Phase 1 · Foundation
Tasks 1–5 · DONE ✓

→ **R1 Setup**
Django + DRF + pytest
health API

→ **R2 Workflow model**
table, unique name,
model tests

→ **R3 Validator**
R3.1 dict + tasks
R3.2 each task
R3.3 names + deps

→ **R4 Kahn's**
R4.1 paper
R4.2 function
R4.3 reject cycles

→ **R5 Workflow API**
R5.1 serializer
R5.2 uses validator
R5.3 create + list
R5.4 detail

▼ you are here

Phase 2 · Run it
Tasks 6–11 · IN PROGRESS

→ **R6.1 Run model**
workflow link,
status, trigger

→ **R6.2 TaskRun model**
status, attempt,
depends_on

→ **R6.3 Create TaskRuns**
start-run API

→ **R6.4 Ready tasks**
which tasks can
run now?

→ **R6.5 Registry**
fn name → Python
function

→ **R6.6 Run one task**
update status
after running

→ **R6.7 Scheduler**
one full cycle

→ **R6.8 Run status**
SUCCESS / FAILED

→ **R6.9 First full run**
workflow runs
end to end

Phase 3 · Parallel
Tasks 12–15 · LATER

→ **Docker Compose**
PostgreSQL + Redis

→ **Celery**
background workers

→ **Safe claiming**
no double-pick
(SKIP LOCKED)

→ **Many workers**
parallel run

Phase 4 · Reliability
Tasks 16–22 · LATER

→ **Attempt logs**
every try saved

→ **Retry + backoff**
wait, then retry

→ **Timeout**
stop stuck tasks

→ **Dead-letter queue**
give up safely

→ **Heartbeat**
worker is alive

→ **Reaper**
recover dead
worker tasks

→ **Idempotency**
twice = once

Phase 5 · Control
Tasks 23–27 · LATER

→ **Cancel**
stop a run

→ **Cron**
run on schedule

→ **Priority**
important first

→ **Parallel limit**
do not overload

→ **Metrics**
success rate,
latency

Phase 6 · Dashboard
Tasks 28–31 · LATER

→ **React setup**
project + API calls

→ **Runs list**
filter by status

→ **Run detail**
DAG picture,
colours

→ **Other pages**
workflows, DLQ,
home

Phase 7 · Proof
Tasks 32–36 · LATER

→ **Chaos tests**
kill a worker,
nothing lost

→ **Load test**
real numbers

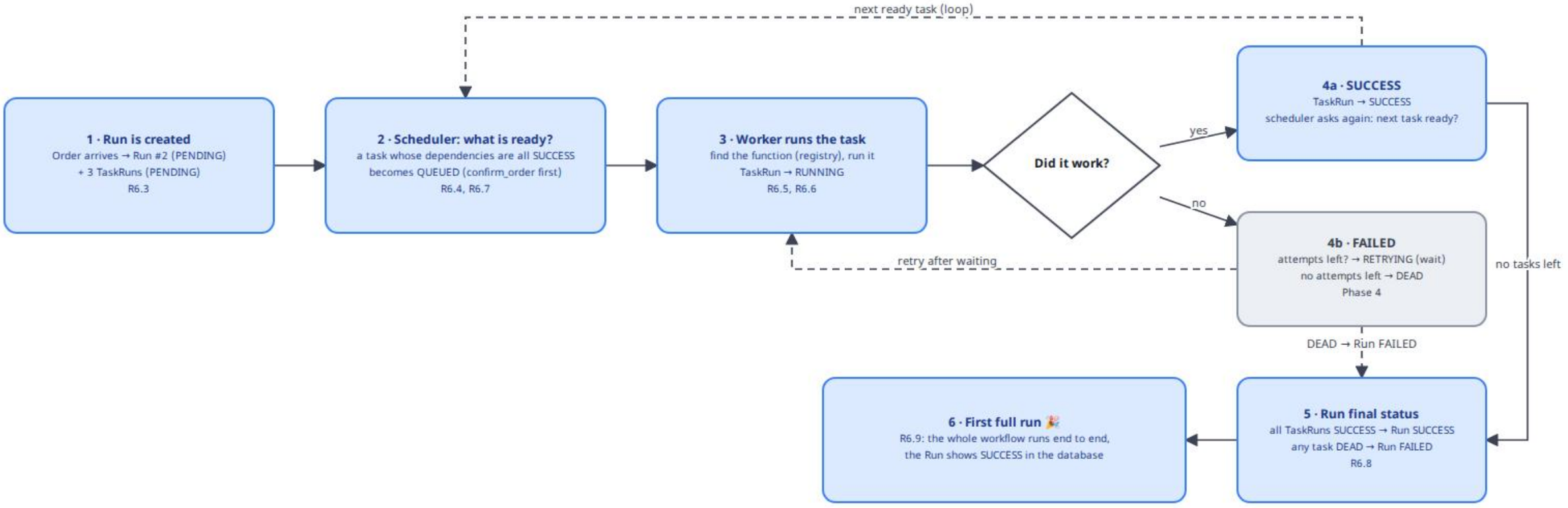
→ **README + CI**
diagram, tests on push

→ **Deploy**
live link

→ **First client**
service website
calls the engine

Run logic — what happens when one order arrives (Sita's order)

Workflow = recipe (confirm → invoice → message). Run = cooking it once. TaskRun = each step of that cooking.



What the database looks like at each moment (Run #2 and its three TaskRuns)

Moment	Run #2	confirm_order	make_invoice	send_message
Run #2 created	PENDING	PENDING	PENDING	PENDING
confirm_order starts	RUNNING	RUNNING	PENDING	PENDING
confirm_order done	RUNNING	SUCCESS	QUEUED	PENDING
make_invoice done	RUNNING	SUCCESS	SUCCESS	QUEUED
send_message fails once	RUNNING	SUCCESS	SUCCESS	RETRYING (try 1 of 3)
send_message works	SUCCESS	SUCCESS	SUCCESS	SUCCESS

How the tickets connect

R6.1–R6.2 build the tables that remember everything.
R6.3 fills them when a Run starts.
R6.4 + R6.7 decide what runs next.
R6.5 + R6.6 actually run it.
R6.8 closes the Run.
R6.9 proves it all works together.

Phase 4 adds retry, timeout, heartbeat and idempotency to the red 'FAILED' path.